

ROS2 TF2

좌표 변환 시스템 · Broadcaster · Listener

cchyun@gmail.com

Part 1. TF2 필요성과 좌표계 개념

좌표계(Frame)란 무엇인가?

- 좌표계(Coordinate Frame)는 로봇이 세상을 바라보는 '관점' 또는 '기준점'
 - 로봇은 단일 객체가 아니라 여러 부품과 센서의 집합체
 - 각각의 위치와 방향을 정의할 기준이 필요
 - 로봇 베이스(`base_link`): 로봇의 회전 중심이나 물리적 중심 기준
 - 카메라(`camera_link`): 렌즈 중심 기준으로 이미지 생성
 - 카메라에서 "앞 1m" = 로봇 바퀴 기준 "위 20cm, 앞 1.1m"
 - LiDAR(`laser_frame`): 레이저 발사 센서 본체 중심 기준
 - 센서가 로봇 중심보다 10cm 앞에 있다면, 1m 측정값 = 로봇 중심 기준 1.1m
 - 팔 끝단(End-effector): 관절 각도를 고려하여 손가락 끝 위치 계산 필요

TF2가 없으면 어떤 문제가 생기는가?

- TF2 없이 개발자가 모든 변환 과정을 직접 코딩해야 하는 어려움
 - 복잡한 삼각함수 계산
 - 로봇이 회전할 때마다 $\sin$, $\cos$ 을 이용한 회전 행렬(Rotation Matrix) 계산 필요
 - 3차원 공간에서는 계산이 기하급수적으로 복잡해짐
 - 오차 누적과 행렬 연산
 - 여러 센서 좌표 통합 시 행렬 곱셈 반복 → 실수 확률 증가, 디버깅 매우 어려움
 - 데이터 동기화 문제
 - 센서 데이터가 들어온 시점 vs. 로봇이 움직인 시점의 미세한 시간 차이를 일일이 계산해서 반영 필요

- TF2(Transform version 2): 로봇 수많은 좌표계 사이 관계를 관리하는 라이브러리 (1/2)
 - 라이브러리 정의 및 역할
 - 표계 관리 표준: ROS 2 시스템 내 수많은 좌표계(Frame) 간의 기하학적 관계를 관리하는 표준 라이브러리
 - 연산 자동화: 복잡한 좌표 변환 행렬 연산을 추상화하여, 사용자 쿼리에 따른 상대 좌표를 즉시 계산
 - 좌표 변환 트리(Transform Tree)
 - 계층적 관리: 모든 좌표계는 부모-자식(Parent-Child) 관계를 가지는 트리 구조로 연결
 - 동적 동기화: 부모 프레임(예: Robot Base) 이동 시, 종속된 모든 자식 프레임(예: Camera, LiDAR)이 상대 거리를 유지하며 동시 이동
 - 방향성 비순환 그래프(DAG): 트리 내 임의의 두 좌표계 간 연결 경로가 존재할 경우, 즉시 변환 행렬 도출 가능

- TF2(Transform version 2): 로봇 수많은 좌표계 사이 관계를 관리하는 라이브러리 (2/2)
 - 리스너 기반 시계열 버퍼 (Time Buffer)
 - 분산형 메모리 저장: 변환 데이터를 수신하는 각 리스너(Listener) 노드의 RAM 내부에 독립적으로 생성 및 관리
 - 이력 보관: 별도 설정이 없는 경우, 기본적으로 과거 10초 동안의 변환 기록을 버퍼에 저장
 - 시계열 역추적: 과거 시점의 타임스탬프를 지정하여 "N초 전 센서 데이터의 현재 로봇 기준 위치" 등을 계산 가능
 - 데이터 보간 기술 (Interpolation)
 - 연속성 확보: 불연속적으로 입수되는 변환 데이터 사이의 공백을 수학적으로 추정하여 계산함
 - 위치(Translation): 선형 보간(Linear Interpolation) 적용
 - 회전(Rotation): 쿼터니언 기반의 구면 선형 보간(SLERP) 기술을 적용하여 부드러운 회전 변환 구현

- ROS 커뮤니티에서 협업을 위해 좌표계 이름을 표준화
 - 글로벌 고정 좌표계 (Global Frames)
 - map (전역 기준점)
 - 정의: 지도의 원점이자 로봇이 최종적으로 맞춰야 할 절대적인 기준점
 - odom (주행 기준점)
 - 정의: 로봇의 센서(인코더, IMU)만을 이용해 계산한 상대적인 이동 궤적
 - 특징: 계산 값이 끊임 없이 부드럽지만(연속성), 시간이 흐를수록 실제 위치와 멀어지는 오차(Drift)가 필연적으로 발생함
 - 로봇 본체 및 센서 좌표계 (Local Frames)
 - base_link: 로봇의 물리적 중심점. 모든 센서 좌표계의 부모 노드.
 - base_footprint: base_link를 바닥면으로 수직 투영한 좌표계 ($Z = 0$). 주로 2D 경로 계획 및 장애물 회피에 활용.
 - camera_optical_frame: 컴퓨터 비전(OpenCV 등) 전용 좌표계. (Z : 렌즈 정면, X : 우측, Y : 하단)
 - laser_frame / base_scan: LiDAR 센서의 원점.

- 로봇 이동 플랫폼에서 좌표계 구성 방법을 정의한 표준

- 권장 계층 구조: `map` → `odom` → `base_link`

- Map과 Odom의 차이

- `odom`: 로봇이 부드럽게 움직이는 것을 보장하지만 시간이 지나면 오차 누적
- `map`: SLAM 같은 알고리즘으로 오차를 주기적으로 보정하여 절대 위치 확보

- 시각적 계층 구조

```
map
├── odom
│   ├── base_link
│   │   ├── camera_link
│   │   └── laser_frame (base_scan)
```


- 처음 TF2를 접할 때 자주 겪는 혼란들

- 화살표의 방향

- RViz2에서 좌표계 사이 관계를 보여주는 화살표는 **자식** → **부모 방향**인 경우가 많음
- 처음에는 반대로 느껴질 수 있으니 주의

- 정적(Static) vs 동적(Dynamic) 변환

- **Static Transform**: 카메라처럼 나사로 고정된 부품 → 한 번만 발행
- **Regular Broadcaster**: 바퀴나 로봇 몸체처럼 계속 움직이는 것 → 주기적으로 발행

- 쿼터니언(Quaternion)

- TF2는 방향(회전)을 **x, y, z, w** 네 개의 숫자로 표현
- 오일러 각도(Roll, Pitch, Yaw)의 **짐벌 락(Gimbal Lock)** 문제를 방지

- TurtleBot3가 방 안을 돌아다니며 벽을 감지하는 상황

- 세상의 중심 (`map`): 방의 한쪽 구석이 (0, 0, 0). 지도는 변하지 않음
 - 출발점 (`odom`): 로봇 전원을 켜 위치
 - 바퀴 미끄러짐(Wheel Slip) → 로봇은 (1, 0)이라고 생각하지만 실제로는 (0.9, 0)
 - 로봇의 중심 (`base_link`): TurtleBot3 몸체 중심. 이동 시 `odom` 내 좌표 변경
 - 벽 탐지 (`laser_frame`): LiDAR 센서가 "내 기준으로 앞 50cm에 벽이 있다"
-
- TF2의 좌표 통합 역할:
 - `laser_frame` 정보(앞 50cm) → `base_link` 기준으로 변환
 - → `odom` 기준으로 변환 → 최종적으로 `map` 기준 위치 계산
 - 결과: "방의 구석(`map` 원점)으로부터 x=2.5m 지점에 벽이 있다"

Part 2. TF2 CLI 도구 실습

- TF2 트리를 시각화하고 디버깅하기 위한 필수 도구들

- `tf2_tools`: TF 트리 전체 구조를 PDF로 저장(`view_frames`)하거나, 두 프레임 간 실시간 변환값을 출력(`tf2_echo`)하는 CLI 도구
- `tf_transformations`: 오일러 각도 ↔ 쿼터니언 변환 등 유틸리티 함수 제공
- `rqt_tf_tree`: GUI 기반 실시간 TF 트리 뷰어

- 설치 명령어

```
sudo apt install ros-humble-tf2-tools ros-humble-tf-transformations  
sudo apt install ros-humble-rqt-tf-tree
```

- `static_transform_publisher`: 좌표계 간 고정된 변환 관계를 발행
 - 로봇에 볼트로 고정된 센서(카메라, LiDAR)나 주행 기준점(`odom`) 같이 상대 위치가 변하지 않는 프레임 관계를 정의
 - `/tf_static` 토픽에 한 번만 발행되며, rqt_tf_tree나 RViz2에서 지속적으로 표시됨
- 명령어 형식

```
# x y z yaw pitch roll frame_id child_frame_id
ros2 run tf2_ros static_transform_publisher 2.0 0.0 0.0 0 0 0 map odom
```

- ``map``을 부모로, ``odom``을 자식으로 두고, x=2.0m, 나머지는 0인 변환 관계 발행

- `tf2_echo`: 두 프레임 사이의 현재 변환값을 실시간으로 출력
 - 브로드캐스터가 잘 작동하는지 즉시 확인할 수 있는 대표적인 디버깅 도구
 - Translation(x,y,z)과 Rotation(x,y,z,w)을 터미널에서 지속적으로 표시
- 실행 예시

```
# 터미널 2: map → odom 변환 확인  
ros2 run tf2_ros tf2_echo map odom
```

- `rqt_tf_tree`: 실행 중인 모든 TF 관계를 그래프로 확인
 - 노드와 화살표로 부모-자식 관계를 시각화하여, 누락된 프레임이나 잘못된 연결을 한눈에 파악
 - 실행 예시

```
# 터미널 3  
ros2 run rqt_tf_tree rqt_tf_tree
```

- 앞서 만든 `map → odom` 아래에 로봇 본체와 센서 좌표계를 추가
 - `odom → base_link`: 로봇의 물리적 중심. 실제로는 주행 데이터(엔코더/IMU)로 동적 발행되지만, 여기서는 고정 값으로 시뮬레이션
 - `base_link → camera_link`: 카메라 마운트 위치. 로봇 중심보다 10cm 앞, 20cm 위에 고정
- 실행 예시

```
# 터미널 4: odom → base_link (고정, 주행 로봇)
ros2 run tf2_ros static_transform_publisher 1.0 0.0 0.0 0 0 1.2 odom base_link

# 터미널 5: base_link → camera_link (고정, 센서 마운트 위치)
ros2 run tf2_ros static_transform_publisher 0.1 0.0 0.2 0 0 0 base_link camera_link
```


- RViz2에서 TF 트리를 3D 공간에 시각화
 - 왼쪽 **Displays** 패널 상단의 **Fixed Frame**을 `map` (또는 `odom`)으로 설정
 - **Add** → **TF** 디스플레이 추가 시, 각 프레임의 축(X=빨강, Y=초록, Z=파랑)이 3D 뷰에 표시됨
 - `map → odom → base_link → camera_link` 체인이 실제 공간 좌표처럼 렌더링되어 직관적 이해 가능
- 실행 예시

```
# 터미널 6  
rviz2
```

- CLI 실습을 통해 확인한 핵심 내용

- `static_transform_publisher`로 고정된 프레임 관계를 한 줄 명령어로 등록 가능
- `tf2_echo`와 `rqt_tf_tree`로 발행된 변환이 올바른지 실시간 검증 가능
- `map → odom → base_link → camera_link` 계층 구조가 실제 로봇 시스템의 좌표계 표준과 동일함을 확인
- RViz2를 통해 2D/3D 공간에서 좌표계 상대 위치를 직관적으로 인식 가능

Part 3. TF2 Broadcaster

/ Listener 구현

실습 1: 표준 TF 트리 구축 — map → odom → base_link → camera_link

- ROS2 로봇의 표준 4계층 TF 계층 구조를 직접 구축하고, 동적 변환까지 체감

- 실습 목표

- `odom → base_link` 동적 변환 노드 작성
- 원 운동을 통한 로봇 주행 시뮬레이션 구현
- `tf2_ros.TransformBroadcaster` 를 활용한 주기적 변환 발행

새 패키지 tf_tutorial_pkg 생성

```
# Python 패키지 생성
cd src
ros2 pkg create --build-type ament_python tf_tutorial_pkg

mkdir -p tf_tutorial_pkg/launch
```

- TF2에서 변환 정보를 전송할 때 사용하는 표준 메시지 구조

- `header.stamp`: 변환이 발생한 시점의 타임스탬프
- `header.frame_id`: 부모 프레임의 이름 (예: `odom`, `map`)
- `child_frame_id`: 자식 프레임의 이름 (예: `base_link`)
- `transform.translation`: 부모 → 자식 프레임까지의 직선 거리 (x, y, z)
- `transform.rotation`: 부모 대비 자식 프레임의 회전 상태 → 쿼터니언(x, y, z, w) 형식

- 주기적 브로드캐스트 패턴

- 오도메트리처럼 시간에 따라 변하는 데이터는 타이머 콜백 내에서 주기적으로 (30~50Hz) `sendTransform()` 호출
- `/tf` 토픽에 발행

쿼터니언(Quaternion) 이해

- TF2가 3D 회전 표현에 쿼터니언을 사용하는 이유
 - 오일러 각도의 문제: 짐벌 락(Gimbal Lock)
 - Roll, Pitch, Yaw 세 개의 각도로 회전 표현
 - 특정 축이 겹치면 **회전 자유도를 잃는** 현상 발생
 - 쿼터니언의 장점
 - x, y, z, w 네 개의 숫자로 3D 공간 회전을 연속적이고 매끄럽게 계산
 - 짐벌 락 문제를 근본적으로 방지
 - `tf_transformations` 라이브러리 활용
 - 입문자가 이해하기 쉬운 오일러 각도 → TF2가 요구하는 쿼터니언 형식으로 변환
 - `euler_to_quaternion()` / `quaternion_from_euler()` 함수 사용

- 고정된 위치의 센서 마운트 좌표를 한 번만 발행하는 브로드캐스터

- 동적 브로드캐스터와의 차이점

구분	TransformBroadcaster	StaticTransformBroadcaster
토픽	/tf	/tf_static
발행 방식	주기적으로 반복	단 한 번만 발행
데이터 유지	계속 갱신 필요	래치(Latched) 되어 유지

- 사용 예시

- 카메라, LiDAR처럼 `base_link`에 고정 장착된 센서의 마운트 좌표 정의
- 주기적인 통신 오버헤드 절감

1-1. odom → base_link 동적 변환 노드 작성

```
src/tf_tutorial_pkg/tf_tutorial_pkg/odom_simulator.py
```

```
import rclpy
from rclpy.node import Node
import tf2_ros
from tf_transformations import quaternion_from_euler
from geometry_msgs.msg import TransformStamped
import math

class OdomSimulator(Node):
    def __init__(self):
        super().__init__('odom_simulator')
        self.br = tf2_ros.TransformBroadcaster(self)
        self.timer = self.create_timer(0.05, self.timer_callback) # 20Hz
        self.radius = 1.0 # 원 반경 (m)
        self.omega = 0.5 # 각속도 (rad/s)
        self.start_time = self.get_clock().now()
```

1-1. odom → base_link 동적 변환 노드 작성

```
def timer_callback(self):
    now = self.get_clock().now()
    t = (now - self.start_time).nanoseconds / 1e9

    # 원 운동: 로봇이 odom 좌표계 중심을 돌며 주행
    x = self.radius * math.cos(self.omega * t)
    y = self.radius * math.sin(self.omega * t)
    # 접선 방향이 yaw (이동 방향)
    roll = 0.0
    pitch = 0.0
    yaw = self.omega * t + math.pi / 2

    qx, qy, qz, qw = quaternion_from_euler(roll, pitch, yaw)

    trans = TransformStamped()
    trans.header.stamp = now.to_msg()
    trans.header.frame_id = 'odom'
    trans.child_frame_id = 'base_link'
    trans.transform.translation.x = x
    trans.transform.translation.y = y
    trans.transform.translation.z = 0.0
    trans.transform.rotation.x = qx
    trans.transform.rotation.y = qy
    trans.transform.rotation.z = qz
    trans.transform.rotation.w = qw

    self.br.sendTransform(trans)

def main(args=None):
    ...
```

1-2. setup.py 엔트리 포인트 등록 / 1-3. package.xml 의존성

- `setup.py`에 엔트리 포인트 추가

```
entry_points={
    'console_scripts': [
        'odom_sim = tf_tutorial_pkg.odom_simulator:main',
    ],
},
```

- `package.xml` 의존성 확인

```
<depend>tf2_ros</depend>
<depend>geometry_msgs</depend>
```

1-4. 빌드 및 실행

```
# 터미널 4
pip3 install transforms3d
colcon build --packages-select tf_tutorial_pkg
source install/setup.bash

# odom → base_link (동적, 로봇 주행 시뮬레이션)
ros2 run tf_tutorial_pkg odom_sim
```

- RViz2에서 `base_link`가 원을 그리며 이동하는지 확인
 - `tf2_echo odom base_link`로 변환값 실시간 검증
 - `rqt_tf_tree`로 트리 구조 확인

실습 2: TF Listener 노드 작성

- `tf_tutorial_pkg`에 새 노드를 추가하여, `base_link` → `camera_link` 변환을 주기적으로 조회
 - 실습 목표
 - `tf2_ros.Buffer`와 `TransformListener` 초기화
 - `lookup_transform()`으로 고정 변환값을 실시간 출력
 - 실습 1의 TF 트리 위에 Listener만 추가로 실행

TransformListener + Buffer 구현

- TF 메시지를 수신·저장하고 변환 정보를 조회하는 리스너

- 초기화 순서

- `tf2_ros.Buffer` 객체 먼저 생성
 - Buffer를 인자로 받는 `tf2_ros.TransformListener` 생성
 - 리스너는 네트워크상의 TF 메시지를 수신하여 버퍼에 **최대 10초간 저장**

- `lookup_transform()` 호출 형식

```
buffer.lookup_transform(target_frame, source_frame, time)
```

- **target_frame**: 기준 좌표계
 - **source_frame**: 대상 좌표계
 - `rclpy.time.Time()` 전달 시 → **가장 최신 변환 반환**

- 발생 가능한 예외

- `LookupException`: 요청한 프레임이 존재하지 않거나 연결되지 않음
 - `ExtrapolationException`: 요청 시점의 데이터가 버퍼에 없음 (너무 과거이거나 미래)

try/except 패턴을 사용하는 이유

- 실시간 제어 루프에서 안전한 변환 조회 방법
 - `wait_for_transform` 방식
 - 변환이 올 때까지 스레드를 차단(Blocking)
 - 제어 루프가 멈출 수 있어 실시간 제어에 부적합
 - `try/except` 방식 (권장)
 - 변환이 없으면 즉시 건너뛰고 다음 루프 실행
 - 실시간 제어 루프에서 응답성 유지

```
try:
    transform = self.tf_buffer.lookup_transform(
        'map', 'camera_link', rclpy.time.Time()
    )
    # 변환 성공 시 처리
except Exception as e:
    self.get_logger().warn(f'Could not transform: {e}')
    # 변환 실패 시 건너뛰
```

2-1. tf_listener.py 작성

```
src/tf_tutorial_pkg/tf_tutorial_pkg/tf_listener.py
```

```
import rclpy
from rclpy.node import Node
import tf2_ros
from geometry_msgs.msg import TransformStamped

class TfListener(Node):
    def __init__(self):
        super().__init__('tf_listener')
        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)
        self.timer = self.create_timer(1.0, self.timer_callback)

    def timer_callback(self):
        try:
            trans = self.tf_buffer.lookup_transform(
                'base_link',      # target_frame
                'camera_link',    # source_frame
                rclpy.time.Time()) # 가장 최근 시간
            t = trans.transform.translation
            self.get_logger().info(
                f'camera_link in base_link: x={t.x:.3f}, y={t.y:.3f}, z={t.z:.3f}'
            )
        except tf2_ros.LookupException as e:
            self.get_logger().warn(f'TF 조회 실패: {e}')
        except tf2_ros.ExtrapolationException as e:
            self.get_logger().warn(f'시간 불일치: {e}')

def main(args=None):
    ...
```


2-2. setup.py / 2-3. package.xml / 2-4. 빌드 및 실행

- `setup.py` 엔트리 포인트 추가

```
entry_points={
    'console_scripts': [
        'odom_sim = tf_tutorial_pkg.odom_simulator:main',
        'tf_listener = tf_tutorial_pkg.tf_listener:main',
    ],
},
```

- `package.xml` 의존성 확인

```
<depend>tf2_ros</depend>
<depend>geometry_msgs</depend>
```

- 빌드 및 실행

```
# 터미널 7
colcon build --packages-select tf_tutorial_pkg
source install/setup.bash

ros2 run tf_tutorial_pkg tf_listener
```



- TF2 사용 시 흔히 발생하는 실수 두 가지
 - 중복 소스(Multiple Sources)
 - 서로 다른 두 노드가 동일한 부모-자식 변환을 발행하는 경우
 - 예: 두 노드가 모두 `map` → `odom` 발행
 - TF 트리 충돌 → 좌표값이 튀는 현상(Jitter) 발생
 - 해결: 각 변환 관계는 하나의 노드에서만 발행
 - 타임스탬프 오류
 - 실제 시간과 시뮬레이션 시간(`use_sim_time`)이 섞이는 경우
 - 타임스탬프를 갱신하지 않고 고정된 값을 보내는 경우
 - 두 경우 모두 변환을 찾지 못하는 오류 발생